Avance 2 - FleetLogix: Análisis SQL y Optimización

Contexto

Con la base de datos poblada con 505.569 registros, este avance consiste en ejecutar 12 queries SQL para resolver problemas operativos reales de FleetLogix, analizar los planes de ejecución con EXPLAIN ANALYZE, y optimizar la performance creando 5 índices estratégicos midiendo la mejora obtenida.

Queries Básicas

Query 1 - Contar vehículos por tipo

Complejidad: Básica
Problema de negocio: Muestra la distribución de la flota por tipo de vehículo, permitiendo planificar compras o retiros según la composición actual.

SQL:

```sql
SELECT vehicle_type, COUNT(*) AS cantidad
FROM vehicles
GROUP BY vehicle_type
ORDER BY cantidad DESC;
```

Resultado:

vehicle_type	cantidad
Van	69
Camión Grande	60
Camión Mediano	51
Motocicleta	20

EXPLAIN ANALYZE:

- Planning Time: 0.100 ms
- Execution Time: 0.124 ms

Query 2 - Conductores con licencia próxima a vencer

Complejidad: Básica
Problema de negocio: Identifica conductores cuya licencia vence en los próximos 30 días para gestionar renovaciones antes de que generen problemas legales u operativos.

SQL:

```
SELECT driver_id, first_name, last_name, license_expiry
FROM drivers
WHERE license_expiry BETWEEN CURRENT_DATE AND CURRENT_DATE + 30
ORDER BY license_expiry ASC;
```

Resultado:

driver_id	first_name	last_name	license_expiry
27	Lorena	Díaz	2026-04-28
65	Camilo	Álvarez	2026-04-28
250	Joan	Muñoz	2026-05-02
239	David	Espitia	2026-05-06
338	Marlon	González	2026-05-06
102	Elizabeth	Bermúdez	2026-05-11

EXPLAIN ANALYZE:

- Planning Time: 0.108 ms
- Execution Time: 0.106 ms

Query 3 - Total de viajes por estado

Complejidad: Básica
Problema de negocio: Muestra cuántos viajes hay en cada estado, permitiendo monitorear en tiempo real si las operaciones fluyen con normalidad.

SQL:

```
SELECT status, COUNT(*) AS total
FROM trips
GROUP BY status;
```

Resultado:

status	total
completed	100.000

EXPLAIN ANALYZE:

- Planning Time: 0.096 ms

- Execution Time: 34.421 ms

Queries Intermedias

Query 4 - Total de entregas por ciudad destino en los últimos 2 meses

Complejidad: Intermedia
Problema de negocio: Revela qué ciudades concentran más entregas en los últimos 2 meses para asignar recursos donde hay mayor demanda.

SQL:

```
SELECT r.destination_city, COUNT(*) AS total_entregas
FROM deliveries d
JOIN trips t ON d.trip_id = t.trip_id
JOIN routes r ON t.route_id = r.route_id
WHERE d.delivered_datetime >= CURRENT_DATE - INTERVAL '2 months'
GROUP BY r.destination_city
ORDER BY total_entregas DESC;
```

Resultado:

destination_city	total_entregas
Bogotá	2356
Medellín	1876
Cali	1752
Barranquilla	1736
Cartagena	1707

EXPLAIN ANALYZE:

- Planning Time: 0.334 ms
- Execution Time: 190.344 ms

Query 5 - Conductores activos con cantidad de viajes completados

Complejidad: Intermedia
Problema de negocio: Muestra la carga de trabajo real de cada conductor activo, útil para detectar sobrecargas o subutilización. *(Se muestran los primeros 15 resultados)*

SQL:

```sql
SELECT d.driver_id, d.first_name, d.last_name, COUNT(t.trip_id) AS
viajes_completados
FROM drivers d
JOIN trips t ON d.driver_id = t.driver_id
WHERE d.status = 'active' AND t.status = 'completed'
GROUP BY d.driver_id, d.first_name, d.last_name
ORDER BY viajes_completados DESC;
```

Resultado:

driver_id	first_name	last_name	viajes_completados
213	Carmen	Blanco	311
137	Yaneth	Castrillón	311
317	Luz	Castro	308
92	Gabriela	Vargas	302
107	Antonio	Rentería	302
37	Humberto	Cerón	302
327	Tania	Calderón	302
262	Dairo	Zapata	302
347	Diego	Barrios	301
50	Alberto	Rodríguez	300
57	Nancy	Garcés	299
243	Luz	Perea	299
7	Sandra	Salamanca	298
182	Aurora	García	298
10	Nohora	Ariza	296

EXPLAIN ANALYZE:

- Planning Time: 0.260 ms
- Execution Time: 53.422 ms

Query 6 - Promedio de entregas por conductor en los últimos 6 meses

Complejidad: Intermedia
Problema de negocio: Mide la productividad individual promedio de cada conductor en los últimos 6 meses como base para evaluaciones de desempeño. *(Se muestran los primeros 15 resultados)*

SQL:

```sql
SELECT d.driver_id, d.first_name, d.last_name,
       ROUND(AVG(entregas_por_viaje), 2) AS promedio_entregas
FROM drivers d
JOIN trips t ON d.driver_id = t.driver_id
JOIN (
    SELECT trip_id, COUNT(*) AS entregas_por_viaje
    FROM deliveries
    GROUP BY trip_id
) sub ON t.trip_id = sub.trip_id
WHERE t.departure_datetime >= CURRENT_DATE - INTERVAL '6 months'
GROUP BY d.driver_id, d.first_name, d.last_name
ORDER BY promedio_entregas DESC;
```

Resultado:

driver_id	first_name	last_name	promedio_entregas
51	Juan	Rodríguez	4.40
110	Mauricio	Díaz	4.40
161	Alexander	Torres	4.38
86	Mery	Ruiz	4.38
2	Ana	Barrios	4.37
306	Estella	Ramos	4.36
27	Lorena	Díaz	4.36
93	Manuel	Díaz	4.33
303	Katherin	Pineda	4.33
294	Lucía	Pérez	4.33
274	Manuel	Núñez	4.32
364	Fabián	Díaz	4.30
320	Dora	Castillo	4.30
192	Alberto	Ochoa	4.29
107	Antonio	Rentería	4.29

EXPLAIN ANALYZE:

- Planning Time: 0.418 ms
- Execution Time: 206.955 ms

Query 7 - Rutas con mayor consumo de combustible por kilómetro

Complejidad: Intermedia
Problema de negocio: Identifica las rutas con mayor consumo de combustible por kilómetro para priorizar optimizaciones de recorrido o mantenimiento.

SQL:

```
SELECT r.route_code, r.origin_city, r.destination_city,
       ROUND(AVG(t.fuel_consumed_liters / r.distance_km), 4) AS consumo_por_km
FROM trips t
JOIN routes r ON t.route_id = r.route_id
WHERE r.distance_km > 0
GROUP BY r.route_id, r.route_code, r.origin_city, r.destination_city
ORDER BY consumo_por_km DESC
LIMIT 10;
```

Resultado:

route_code	origin_city	destination_city	consumo_por_km
R009	Bogotá	Barranquilla	0.1160
R016	Medellín	Cali	0.1158
R037	Barranquilla	Cali	0.1158
R011	Bogotá	Cartagena	0.1156
R040	Cartagena	Bogotá	0.1155
R003	Bogotá	Medellín	0.1155
R045	Cartagena	Cali	0.1155
R035	Barranquilla	Medellín	0.1154
R025	Cali	Medellín	0.1154
R005	Bogotá	Cali	0.1154

EXPLAIN ANALYZE:

- Planning Time: 0.219 ms
- Execution Time: 78.231 ms

Query 8 - Análisis de entregas retrasadas por día de la semana

Complejidad: Intermedia
Problema de negocio: Detecta qué días de la semana concentran más retrasos, permitiendo ajustar planificación y dotación de personal.

SQL:

```sql
SELECT
    TO_CHAR(d.scheduled_datetime, 'Day') AS dia_semana,
    EXTRACT(DOW FROM d.scheduled_datetime) AS num_dia,
    COUNT(*) AS total_entregas,
    SUM(CASE WHEN d.delivered_datetime > d.scheduled_datetime + INTERVAL '30
minutes'
        THEN 1 ELSE 0 END) AS entregas_retrasadas,
    ROUND(100.0 * SUM(CASE WHEN d.delivered_datetime > d.scheduled_datetime +
INTERVAL '30 minutes'
        THEN 1 ELSE 0 END) / COUNT(*), 2) AS pct_retraso
FROM deliveries d
WHERE d.delivery_status = 'delivered'
GROUP BY dia_semana, num_dia
ORDER BY num_dia;
```

Resultado:

dia_semana	num_dia	total_entregas	entregas_retrasadas	pct_retraso
Sunday	0	56970	5599	9.83
Monday	1	57469	5734	9.98
Tuesday	2	57158	5545	9.70
Wednesday	3	57299	5680	9.91
Thursday	4	57115	5643	9.88
Friday	5	56875	5672	9.97
Saturday	6	57115	5541	9.70

EXPLAIN ANALYZE:

- Planning Time: 0.195 ms
- Execution Time: 951.524 ms

Queries Complejas

Query 9 - Costo de mantenimiento por kilómetro recorrido

Complejidad: Compleja
Problema de negocio: Calcula el costo de mantenimiento por kilómetro según tipo de vehículo para evaluar cuáles son más rentables a largo plazo.

SQL:

```sql
SELECT v.vehicle_type,
       ROUND(SUM(m.cost), 2) AS costo_total_mantenimiento,
       ROUND(SUM(t.fuel_consumed_liters * r.distance_km), 2) AS km_totales,
       ROUND(SUM(m.cost) / NULLIF(SUM(r.distance_km), 0), 2) AS costo_por_km
FROM vehicles v
JOIN maintenance m ON v.vehicle_id = m.vehicle_id
JOIN trips t ON v.vehicle_id = t.vehicle_id
JOIN routes r ON t.route_id = r.route_id
GROUP BY v.vehicle_type
ORDER BY costo_por_km DESC;
```

Resultado:

vehicle_type	costo_total_mantenimiento	km_totales	costo_por_km
Motocicleta	88.792.754.743,81	13.077.863.686,30	547.84
Van	295.637.286.132,98	43.923.188.680,21	547.24
Camión Grande	246.816.780.527,72	37.031.202.859,13	543.63
Camión Mediano	217.564.010.103,84	32.697.596.559,41	540.64

EXPLAIN ANALYZE:

- Planning Time: 0.475 ms
- Execution Time: 1055.877 ms

Query 10 - Ranking de conductores por eficiencia (Window Functions)

Complejidad: Compleja
Problema de negocio: Rankea conductores por eficiencia combinando entregas realizadas y consumo de combustible para identificar top performers. *(Se muestran los primeros 20 resultados)*

SQL:

```sql
WITH conductor_stats AS (
    SELECT d.driver_id, d.first_name, d.last_name,
           COUNT(t.trip_id) AS total_viajes,
           ROUND(AVG(t.fuel_consumed_liters / NULLIF(r.distance_km, 0)), 4) AS
consumo_promedio,
           COUNT(del.delivery_id) AS total_entregas
    FROM drivers d
    JOIN trips t ON d.driver_id = t.driver_id
    JOIN routes r ON t.route_id = r.route_id
    JOIN deliveries del ON t.trip_id = del.trip_id
    WHERE t.status = 'completed'
    GROUP BY d.driver_id, d.first_name, d.last_name
)
SELECT *,
```

```
        RANK() OVER (ORDER BY total_entregas DESC, consumo_promedio ASC) AS ranking
FROM conductor_stats
ORDER BY ranking
LIMIT 20;
```

Resultado:

driver_id	first_name	last_name	total_viajes	consumo_promedio	total_entregas	ranking
317	Luz	Castro	1264	0.1143	1264	1
213	Carmen	Blanco	1261	0.1157	1261	2
137	Yaneth	Castrillón	1256	0.1157	1256	3
327	Tania	Calderón	1227	0.1153	1227	4
347	Diego	Barrios	1224	0.1173	1224	5
50	Alberto	Rodríguez	1219	0.1166	1219	6
57	Nancy	Garcés	1212	0.1160	1212	7
7	Sandra	Salamanca	1210	0.1168	1210	8
37	Humberto	Cerón	1203	0.1185	1203	9
11	Hernando	Serrano	1202	0.1162	1202	10
262	Dairo	Zapata	1201	0.1152	1201	11
10	Nohora	Ariza	1196	0.1141	1196	12
107	Antonio	Rentería	1191	0.1145	1191	13
92	Gabriela	Vargas	1189	0.1171	1189	14
116	Dario	Rodríguez	1186	0.1155	1186	15
318	Alexandra	Rodríguez	1186	0.1161	1186	16
22	Vanessa	Narváez	1185	0.1156	1185	17
380	David	Torres	1182	0.1161	1182	18
209	Patricia	López	1181	0.1165	1181	19
182	Aurora	García	1177	0.1150	1177	20

EXPLAIN ANALYZE:

- Planning Time: 0.885 ms
- Execution Time: 387.765 ms

Query 11 - Análisis de tendencia de viajes con LAG y LEAD

Complejidad: Compleja
Problema de negocio: Analiza la evolución mensual de viajes comparando cada mes con el anterior y el siguiente para detectar tendencias y proyectar necesidades futuras.

SQL:

```sql
WITH viajes_mensuales AS (
    SELECT
        DATE_TRUNC('month', departure_datetime) AS mes,
        COUNT(*) AS total_viajes
    FROM trips
    GROUP BY mes
)
SELECT
    mes,
    total_viajes,
    LAG(total_viajes) OVER (ORDER BY mes) AS mes_anterior,
    LEAD(total_viajes) OVER (ORDER BY mes) AS mes_siguiente,
    total_viajes - LAG(total_viajes) OVER (ORDER BY mes) AS variacion
FROM viajes_mensuales
ORDER BY mes;
```

Resultado:

mes	total_viajes	mes_anterior	mes_siguiente	variacion
2024-04	3291	-	4464	-
2024-05	4464	3291	4320	+1173
2024-06	4320	4464	4464	-144
2024-07	4464	4320	4464	+144
2024-08	4464	4464	4320	0
2024-09	4320	4464	4464	-144
2024-10	4464	4320	4320	+144
2024-11	4320	4464	4464	-144
2024-12	4464	4320	4464	+144
2025-01	4464	4464	4032	0
2025-02	4032	4464	4464	-432
2025-03	4464	4032	4320	+432
2026-02	4032	4464	373	-432
2026-03	373	4032	-	-3659

EXPLAIN ANALYZE:

- Planning Time: 0.142 ms
- Execution Time: 35.047 ms

Query 12 - Pivot de entregas por hora y día de la semana

Complejidad: Compleja
Problema de negocio: Muestra la distribución de entregas por hora y día de la semana en formato tabla para optimizar horarios operativos y asignación de personal.

SQL:

```sql
SELECT
    EXTRACT(HOUR FROM scheduled_datetime) AS hora,
    SUM(CASE WHEN EXTRACT(DOW FROM scheduled_datetime) = 1 THEN 1 ELSE 0 END) AS
lunes,
    SUM(CASE WHEN EXTRACT(DOW FROM scheduled_datetime) = 2 THEN 1 ELSE 0 END) AS
martes,
    SUM(CASE WHEN EXTRACT(DOW FROM scheduled_datetime) = 3 THEN 1 ELSE 0 END) AS
miercoles,
    SUM(CASE WHEN EXTRACT(DOW FROM scheduled_datetime) = 4 THEN 1 ELSE 0 END) AS
jueves,
    SUM(CASE WHEN EXTRACT(DOW FROM scheduled_datetime) = 5 THEN 1 ELSE 0 END) AS
viernes,
    SUM(CASE WHEN EXTRACT(DOW FROM scheduled_datetime) = 6 THEN 1 ELSE 0 END) AS
sabado,
    SUM(CASE WHEN EXTRACT(DOW FROM scheduled_datetime) = 0 THEN 1 ELSE 0 END) AS
domingo
FROM deliveries
GROUP BY hora
ORDER BY hora;
```

Resultado:

hora	lunes	martes	miercoles	jueves	viernes	sabado	domingo
0	1826	1835	1910	1820	1805	1898	1816
1	1635	1602	1600	1610	1604	1603	1649
2	1406	1452	1404	1431	1454	1409	1458
3	1289	1309	1261	1263	1218	1347	1253
4	1244	1250	1273	1233	1189	1222	1228
5	1196	1190	1156	1176	1149	1171	1183
6	1262	1258	1205	1207	1183	1279	1281
7	1549	1684	1593	1605	1535	1624	1619
8	1933	1912	1865	1901	1763	1855	1876

hora	lunes	martes	miercoles	jueves	viernes	sabado	domingo
9	2278	2286	2280	2315	2296	2249	2261
10	2636	2576	2592	2510	2577	2588	2598
11	2891	2879	2944	2959	2861	2870	2749
12	3130	3064	3106	3072	3140	3161	3104
13	3098	3069	3205	3215	3139	3177	3154
14	3224	3182	3162	3234	3210	3158	3170
15	3355	3261	3293	3284	3267	3351	3212
16	3387	3294	3358	3410	3305	3234	3213
17	3320	3218	3337	3342	3355	3271	3310
18	3270	3235	3229	3253	3265	3237	3255
19	3246	3266	3207	3166	3222	3191	3217
20	3133	3107	3086	2991	3119	3010	3078
21	2718	2681	2704	2620	2691	2688	2659
22	2334	2380	2379	2356	2432	2441	2404
23	2109	2168	2150	2142	2096	2081	2223

EXPLAIN ANALYZE:

- Planning Time: 0.151 ms
- Execution Time: 952.952 ms

Optimización con Índices

Índices creados

```
-- ÍNDICE 1: Optimización para JOINs frecuentes en trips
CREATE INDEX idx_trips_composite_joins ON trips(vehicle_id, driver_id, route_id,
departure_datetime)
WHERE status = 'completed';

-- ÍNDICE 2: Optimización para análisis temporal de deliveries
CREATE INDEX idx_deliveries_scheduled_datetime ON deliveries(scheduled_datetime,
delivery_status)
WHERE delivery_status = 'delivered';

-- ÍNDICE 3: Optimización para mantenimiento por vehículo
CREATE INDEX idx_maintenance_vehicle_cost ON maintenance(vehicle_id, cost);

-- ÍNDICE 4: Optimización para análisis de conductores
```

```sql
CREATE INDEX idx_drivers_status_license ON drivers(status, license_expiry)
WHERE status = 'active';

-- ÍNDICE 5: Optimización para métricas de rutas
CREATE INDEX idx_routes_metrics ON routes(route_id, distance_km,
destination_city);
```

Comparación de tiempos

Query	Antes	Después	Mejora
Q4 - Entregas por ciudad	190.344 ms	114.439 ms	**39.9%**
Q6 - Promedio por conductor	206.955 ms	194.972 ms	**5.8%**
Q7 - Consumo por ruta	78.231 ms	80.058 ms	Sin mejora
Q8 - Retrasos por día	951.524 ms	870.794 ms	**8.5%**
Q9 - Costo por km	1055.877 ms	1279.265 ms	Sin mejora
Q10 - Ranking conductores	387.765 ms	308.006 ms	**20.6%**
Q11 - Tendencia viajes	35.047 ms	37.229 ms	Sin mejora
Q12 - Pivot hora/día	952.952 ms	966.390 ms	Sin mejora

Análisis de resultados

Las mejoras más significativas se observaron en Q4 (39.9%) y Q10 (20.6%), donde los índices sobre `trips` optimizaron los JOINs con `deliveries` y `routes`. Q6 mejoró levemente (5.8%) gracias al índice `idx_trips_departure` que aceleró el filtro por fecha.

Las queries Q7, Q9, Q11, Q12 no mostraron mejora significativa porque realizan full scan sobre tablas completas con ordenamiento en disco — este tipo de operación no se beneficia de índices tradicionales. Para optimizarlas se requeriría particionamiento de tablas o vistas materializadas.

Conclusiones del Avance 2

Se ejecutaron y documentaron las 12 queries SQL de FleetLogix, organizadas por complejidad creciente. Las queries más costosas en tiempo fueron Q8, Q9 y Q12 con tiempos superiores a 950ms, todas operando sobre las tablas `deliveries` y `trips` que concentran el mayor volumen de datos (400.001 y 100.000 registros respectivamente).

La creación de 5 índices estratégicos apunta a reducir estos tiempos mejorando los JOINs más frecuentes y los filtros por fecha y estado.

> **Nota:** La Query 12 registró un tiempo ligeramente superior después de la indexación (966.390 ms vs 952.952 ms). Esto se debe a que la query realiza un ordenamiento externo en disco sobre los 400.001 registros de `deliveries`, operación que los índices no pueden optimizar. El overhead de consultar el índice sumado al sort en disco explica el leve incremento. Para optimizar esta query se requeriría particionamiento por fecha o una vista materializada pre-calculada.